



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.8: Zero-Knowledge Proofs

Nadim Kobeissi

<https://appliedcryptography.page>

Section 1

Introduction

Zero-knowledge proofs: magic, but real

Imagine this scenario:

- I claim to know the password to a secret vault
- You need to verify I really know it
- But I don't want to tell you the password!

Seems impossible?

- How can I prove knowledge without revealing it?
- This is the magic of zero-knowledge proofs

Zero-knowledge property

A proof that reveals **nothing** except the truth of the statement being proven

The paradox

Convince you completely while teaching you nothing

What makes zero-knowledge proofs special?

Traditional proof:

- “Here’s my password: hunter2”
- You learn the secret itself
- You could now impersonate me

Zero-knowledge proof:

- “I’ll convince you I know the password”
- You learn **only** that I know it
- You gain no ability to prove it yourself
- You can’t replay my proof to others

The three zero-knowledge properties

1. Completeness

If the statement is true, an honest prover can convince an honest verifier

2. Soundness

If the statement is false, no cheating prover can convince an honest verifier (except with negligible probability)

3. Zero-Knowledge

The verifier learns nothing beyond the validity of the statement

These three properties together create the “magic”

Also, knowledge soundness

- **Regular Soundness:**

- Prevents false statements from being proved
- “Can’t prove something that’s false”
- Example: Can’t prove $g^a = X$ if no such a exists
- Weaker guarantee

- **Sufficient when:**

- Only care about statement validity
- Don’t need witness extraction
- Simple yes/no verification

- **Knowledge Soundness:**

- If prover succeeds, they *know* a witness^a
- “Can extract the witness from successful prover”
- Example: Can extract the actual value a
- Stronger guarantee

- **Essential when:**

- Building signature schemes
- Proving knowledge credentials
- Composition with other protocols

^aA witness is the secret information or evidence that proves a statement is true - like the private key that corresponds to a public key, or the solution to a puzzle.

Example: Proving Knowledge of a Private Key

The Setup:

- Public key: $A = g^a$
- I claim to know a
- But revealing a would compromise security

Zero-Knowledge Solution:

1. Interactive protocol
2. Multiple rounds of challenges
3. Statistically convincing
4. Reveals nothing about a

Can't prove to others

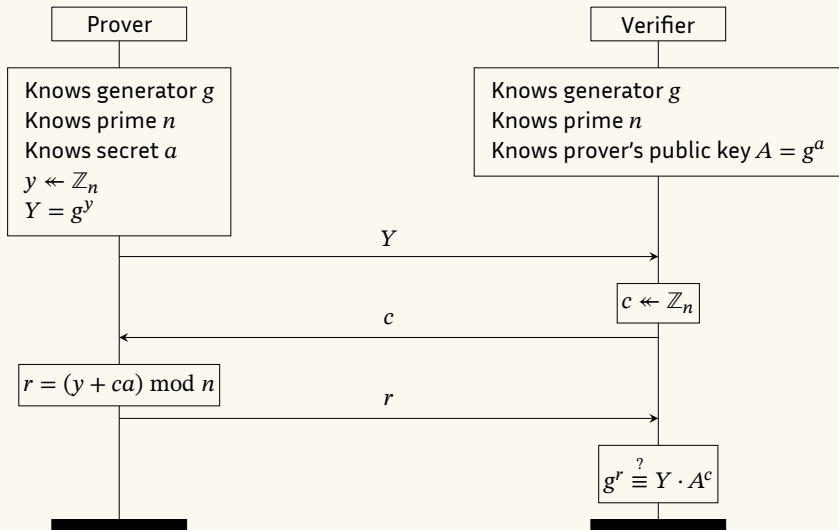
After our interaction:

- You're 100% convinced I know a
- You've learned nothing about a
- You can't prove to others that I know it

No "trace" or "evidence"

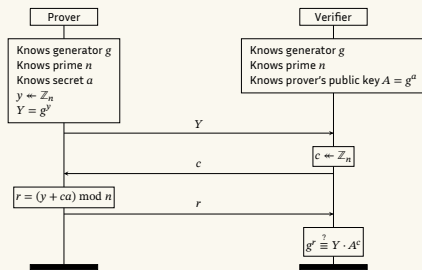
The interaction leaves no transferable evidence

Schnorr Identification Protocol



Schnorr Identification Protocol

- **Interactive proof:** Multiple rounds of communication
- **Commitment phase:** Prover commits to random $Y = g^y$
- **Challenge phase:** Verifier sends random challenge c
- **Response phase:** Prover computes $r = y + ca$
- **Verification:** Check if $g^r = Y \cdot A^c$
- **Zero-knowledge property:** r reveals nothing about a due to random masking by y
- **Soundness:** Without knowing a , prover can't answer random challenges correctly



The verifier learned literally nothing new

- After the protocol, you're convinced I know the secret
- But you can't convince anyone else!
- Why? The transcript looks identical to **something you could have generated alone**
- **The simulation argument:**
 - You could have picked r and c randomly
 - Then computed $Y = g^r / A^c$
 - This produces a valid-looking transcript!
 - No way to distinguish from real interaction

Self-generated transcripts

For any challenge c and response r :

- Set $Y = g^r / A^c$
- Now (Y, c, r) is a valid transcript
- Indistinguishable from real protocol
- Real interaction: Prover knows secret
- Simulated transcript: No secret needed
- Both look exactly the same!
- This is what makes it "zero-knowledge"

In other words...

$\mathcal{L}_{\text{schnorr-real}}$

SCHNORR.TRANSCRIPT(a):

$A := g^a$

$y \leftarrow \mathbb{Z}_n$

$Y := g^y$

$c \leftarrow \mathbb{Z}_n$

$r := (y + ca) \bmod n$

return (A, Y, c, r)

$\equiv$

$\mathcal{L}_{\text{schnorr-fake}}$

SCHNORR.TRANSCRIPT(a):

$A := g^a$

$c \leftarrow \mathbb{Z}_n$

$r \leftarrow \mathbb{Z}_n$

$Y := g^r \cdot (A^c)^{-1}$

return (A, Y, c, r)

Quick reminder: What does x^{-1} mean?

In modular arithmetic:

- x^{-1} is the **modular inverse** of x
- It's the number that, when multiplied by x , gives 1
- $x \cdot x^{-1} \equiv 1 \pmod{n}$

Example:

- If $x = 3$ and $n = 7$
- Then $x^{-1} = 5$ because $3 \cdot 5 = 15 \equiv 1 \pmod{7}$

In group elements:

- If $A = g^a$, then $A^{-1} = g^{-a}$
- $A \cdot A^{-1} = g^a \cdot g^{-a} = g^{a-a} = g^0 = 1$
- It's the element that "cancels out" A

Key property

$$(A^c)^{-1} = A^{-c}$$

This is why we can rewrite expressions!

Proof of indistinguishability

Step 1

- We can rearrange things so that c is chosen first.

$\mathcal{L}_{\text{schnorr-real}}$

SCHNORR.TRANSCRIPT(a):

$A := g^a$

$c \leftarrow \mathbb{Z}_n$

$y \leftarrow \mathbb{Z}_n$

$r := (y + ca) \bmod n$

$Y := g^y$

return (A, Y, c, r)

Proof of indistinguishability

Step 2

- r is essentially an OTP encryption of plaintext ca under key y , using a variant of OTP with addition mod n .
- We can achieve the same distribution by choosing r uniformly and solving for y .

$\mathcal{L}_{\text{schnorr-real}}$

SCHNORR.TRANSCRIPT(a):

$A := g^a$

$c \leftarrow \mathbb{Z}_n$

$r \leftarrow \mathbb{Z}_n$

$y := (r - ca) \bmod n$

$Y := g^y$

return (A, Y, c, r)

Proof of indistinguishability

Step 3

- Y is computed as

$$\begin{aligned} Y &= g^y \\ &= g^{r-ca} \\ &= g^r \cdot (g^{ca})^{-1} \\ &= g^r \cdot (A^c)^{-1} \end{aligned}$$

- By rewriting Y in this way, the private exponent a is no longer needed.

$\mathcal{L}_{\text{schnorr-fake}}$

SCHNORR.TRANSCRIPT(a):

$A := g^a$

$c \leftarrow \mathbb{Z}_n$

$r \leftarrow \mathbb{Z}_n$

$y := (r - ca) \bmod n$

$Y := g^r \cdot (A^c)^{-1}$

return (A, Y, c, r)

Understanding the exponent manipulation

Why does $g^{r-ca} = g^r \cdot (g^{ca})^{-1}$?

- Recall the exponent rule: $g^{x-y} = \frac{g^x}{g^y}$
- In group notation, division is multiplication by inverse
- So $\frac{g^x}{g^y} = g^x \cdot (g^y)^{-1}$

Step by step:

- $g^{r-ca} = \frac{g^r}{g^{ca}}$ (exponent rule)
- $\frac{g^r}{g^{ca}} = g^r \cdot (g^{ca})^{-1}$ (division = multiply by inverse)

Concrete example

Let's use regular numbers to see why:

- If $g = 2, r = 7, ca = 3$
- $2^{7-3} = 2^4 = 16$
- $2^7 \cdot (2^3)^{-1} = 128 \cdot \frac{1}{8} = 16 \checkmark$

Observe how...

This manipulation lets us separate the secret exponent a from the computation!

In other words...

$\mathcal{L}_{\text{schnorr-real}}$

SCHNORR.TRANSCRIPT(a):

$A := g^a$

$y \leftarrow \mathbb{Z}_n$

$Y := g^y$

$c \leftarrow \mathbb{Z}_n$

$r := (y + ca) \bmod n$

return (A, Y, c, r)

$\equiv$

$\mathcal{L}_{\text{schnorr-fake}}$

SCHNORR.TRANSCRIPT(a):

$A := g^a$

$c \leftarrow \mathbb{Z}_n$

$r \leftarrow \mathbb{Z}_n$

$Y := g^r \cdot (A^c)^{-1}$

return (A, Y, c, r)

A parable

“King Richard decides to hold an archery competition to identify the best archer in the land. He constructs a long wooden wall, with many targets painted on it. During the competition, he is stunned as Robin Hood stands 100 meters away from the wall and fires one arrow into the center of each of the targets. Of course, Robin Hood is declared the winner.”

A parable

“Later that day, the Sheriff of Nottingham arrives at the King’s castle. The King describes Robin Hood’s exceptional performance, pointing to the arrows still in the targets. The Sheriff is not impressed. “Robin Hood is a fraud and a liar. Those arrows prove nothing,” he says. He takes his own bow and fires a series of arrows into blank parts of the wall. He finds a bucket of paint and paints a target around each of his arrows. “See,” he says again, “arrows in targets prove nothing!””

Another parable

“Alice, whose public key is $A = g^a$, contacts Bob and uses Schnorr’s protocol to prove that she knows the secret key a . Bob is convinced that he is talking to Alice, and he writes down everything he sees: (A, Y, c, r) .”

Another parable

“Later that day, Charlie arrives at Bob’s house. Bob mentions that he talked to Alice earlier in the day, and he shows Charlie the accepting transcript (A, Y, c, r) . “That proves nothing,” says Charlie, and he proceeds to generate his own accepting transcript (A, Y', c', r') , even though he doesn’t know a .”

Section 2

Sigma Protocols

ZK can scale to entire systems

Reminder

- As seen previously, you can build entire **zero-knowledge protocols**, not just primitives that prove knowledge of certain numbers.
- “Zero-knowledge virtual machines” where you can execute an entire program that runs as a zero-knowledge proof.
- ZKP battleship game: server proves to the players that its output to their battleship guesses is correct, without revealing any additional information (e.g. ship location).



Battleship board game. Source: Hasbro

Beyond authentication: the power of zero-knowledge

Zero-knowledge is not limited to proving identity!

- **Can prove statements like:**

- *"I know a solution to this Sudoku puzzle"*
- *"This encrypted value is in range $[0, 100]$ "*
(range proofs)
- *"I performed this computation correctly"*
- ...without revealing any information!

- **Applications:**

- Private transactions
(Zcash)
- Anonymous credentials
- Secure voting systems
- Privacy-preserving audits

Section 2

Sigma Protocols

Subsection 2.1

Sigma Protocols: High-Level Overview

From Schnorr to Sigma, a general framework

Schnorr was just the beginning!

- Schnorr proves: "I know a such that $g^a = A$ "
- What if we want to prove other statements?
- Enter **Sigma protocols**: a general framework

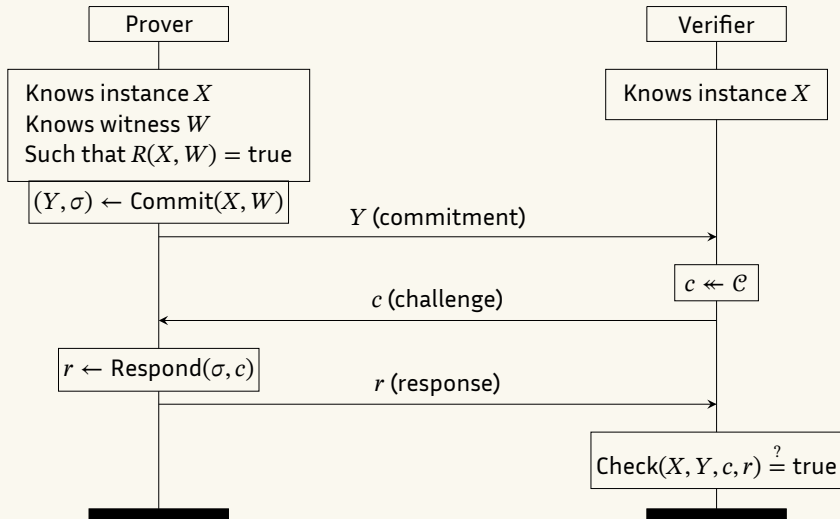
The pattern:

- Public instance: X
- Private witness: W
- Relation: $R(X, W) = \text{true}$

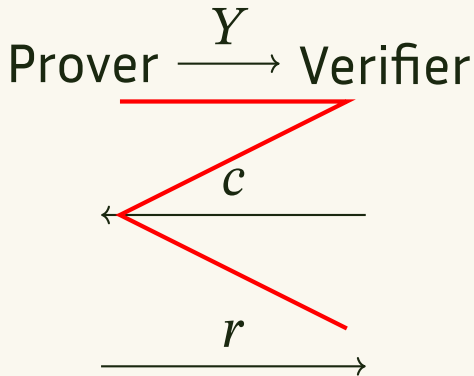
Examples of relations

- "I know a such that $g^a = X$ "
 - Witness: a
- "I know (r, s) such that $g^r h^s = X$ "
 - Witness: (r, s)
- "I know a factorization of X "
 - Witness: (p, q) where $X = p \cdot q$
- "I know a graph coloring for X "
 - Witness: The coloring assignment

Sigma protocol structure



Why “Sigma”? The shape of the protocol



The Greek letter Σ (Sigma)

- Three message flows
- Alternating direction
- Forms a “zigzag” pattern
- Hence: Sigma protocols!

Always this pattern

1. Prover $\rightarrow$ Verifier: Commitment
2. Verifier $\rightarrow$ Prover: Challenge
3. Prover $\rightarrow$ Verifier: Response

Formal definition of Sigma protocols

Definition: Sigma Protocol

A sigma protocol consists of the following parameters:

- **Commit:** A randomized algorithm that takes instance X and witness W as input, outputs commitment Y and state σ
- $\mathcal{C}$: A set of verifier challenges
- **Respond:** A deterministic algorithm that takes state σ and challenge $c \in \mathcal{C}$ as input, outputs response r
- **Check:** A deterministic algorithm that takes transcript (X, Y, c, r) as input, outputs true/false

Example: Schnorr as a Sigma protocol

Instance and Witness:

- Instance: $X = A = g^a$
- Witness: $W = a$
- Relation: $g^W = X$

The Algorithms:

- **Commit**(A, a):
 - $y \leftarrow \mathbb{Z}_n$
 - $Y = g^y$
 - Return $(Y, \sigma = y)$

- $\mathcal{C} = \mathbb{Z}_n$ (all possible challenges)
- **Respond**($\sigma = y, c$):
 - Return $r = (y + ca) \bmod n$
- **Check**(A, Y, c, r):
 - Return $(g^r = Y \cdot A^c)$

Fits the framework!

Schnorr is just one example of a Sigma protocol

Example: proving knowledge of a representation

The problem:

- Given: $X = g^a h^b$
- Prove: "I know (a, b) "
- Without revealing (a, b) !

Why is this useful?

- Pedersen commitments
- Mix networks
- Electronic voting

The Sigma protocol:

- **Commit:**

- Pick random r, s
- $Y = g^r h^s$
- State: $\sigma = (r, s)$

- **Respond:**

- $u = r + ca$
- $v = s + cb$
- Return (u, v)

- **Check:**

- $g^u h^v \stackrel{?}{=} Y \cdot X^c$

The power of Sigma protocols

Sigma protocols can prove almost any statement!

Graph Problems:

- “I know a 3-coloring of this graph”
- “I know a Hamiltonian cycle”
- “I know an isomorphism between graphs”

Number Theory:

- “I know the square root of X mod N ”
- “I know the factorization of N ”

Cryptographic Statements:

- “This ciphertext encrypts 0 or 1”
- “These ciphertexts encrypt the same value”
- “I shuffled these ciphertexts correctly”

Universal result

Any NP statement has a Sigma protocol!

Section 2

Sigma Protocols

Subsection 2.2

Formal Properties of Sigma Protocols

The three essential properties

What makes a Sigma protocol actually work?

Mathematical guarantees:

1. **Completeness:** Honest provers always succeed
2. **Soundness:** Dishonest provers always fail
3. **Zero-Knowledge:** Nothing leaks about the witness

In proof language

- Completeness: Every true thing has a valid proof
- Soundness: Only true things have valid proofs
- Zero-Knowledge: Proofs reveal nothing extra

All three are essential for a useful protocol!

Completeness: honest provers always win

Definition: Completeness

A Sigma protocol is **complete** if:

- For any instance X and witness W where $R(X, W) = \text{true}$
- When the prover follows the protocol honestly
- The verifier *always* accepts

In other words:

- If you actually know the secret, you can always prove it
- No “false negatives” - legitimate users never get rejected
- The protocol doesn’t accidentally lock out honest participants

Example: Schnorr's completeness

Why does Schnorr always work for honest provers?

The honest execution:

- Prover knows a where $A = g^a$
- Sends $Y = g^y$ for random y
- Receives challenge c
- Responds with $r = y + ca$

Verification always succeeds:

$$\begin{aligned} g^r &= g^{y+ca} \\ &= g^y \cdot g^{ca} \\ &= Y \cdot (g^a)^c \\ &= Y \cdot A^c \quad \checkmark \end{aligned}$$

Perfect completeness

Honest prover succeeds with probability 1

Special soundness: cheaters get caught

Unique to Sigma protocols

The challenge for cheaters:

- Prover doesn't know the witness
- Must commit to Y before seeing challenge
- Can't predict what challenge will be
- Must be ready for *any* challenge!

The extraction principle:

- If you can answer two different challenges...
- ...for the same commitment Y ...
- ...then you must know the witness!

Key insight

Being able to handle multiple challenges proves knowledge

Why "special"?

This specific form of soundness is unique to Sigma protocols

Special soundness: formal definition

Definition: Special Soundness

A Sigma protocol has **special soundness** if there exists an efficient algorithm **Extract** such that:

- Given two accepting transcripts: (X, Y, c, r) and (X, Y, c', r')
- With same instance X and commitment Y
- But different challenges $c \neq c'$
- **Extract** (X, Y, c, r, c', r') outputs witness W
- Such that $R(X, W) = \text{true}$

Interpretation: If you can answer two different challenges for the same commitment, you must know the secret!

Example: extracting from Schnorr

Given two accepting transcripts with same Y but different challenges:

Transcript 1:

- Commitment: Y
- Challenge: c
- Response: r
- Valid: $g^r = Y \cdot A^c$

Transcript 2:

- Commitment: Y (same!)
- Challenge: c'
- Response: r'
- Valid: $g^{r'} = Y \cdot A^{c'}$

Extract the secret:

$$g^r = Y \cdot A^c$$

$$g^{r'} = Y \cdot A^{c'}$$

$$\frac{g^r}{g^{r'}} = \frac{A^c}{A^{c'}}$$

$$g^{r-r'} = A^{c-c'}$$

$$g^{r-r'} = g^{a(c-c')}$$

$$r - r' = a(c - c') \pmod{n}$$

$$a = \frac{r - r'}{c - c'} \pmod{n}$$

Why extraction implies soundness

The contrapositive argument:

- If prover doesn't know witness...
- Then can't produce two valid responses
- So can succeed for at most one challenge
- But challenge is random!
- Probability of guessing right: $\frac{1}{|\mathcal{C}|}$

Soundness guarantee

$$\text{Cheating probability} \leq \frac{1}{\text{number of challenges}}$$

For Schnorr

$|\mathcal{C}| = n$ (prime order)
Cheating probability $\approx \frac{1}{2^{256}}$
Essentially impossible!

Zero-knowledge: the formal definition

Definition: Zero-Knowledge

A Sigma protocol is **zero-knowledge** if:

- There exists a simulator algorithm that can generate convincing transcripts
- Without knowing the witness!
- These simulated transcripts are indistinguishable from real ones

In other words:

- If fake transcripts look identical to real ones...
- And fake transcripts contain no information about the witness...
- Then real transcripts must not leak information either!
- We saw this earlier when we proved $\mathcal{L}_{\text{schnorr-real}} \equiv \mathcal{L}_{\text{schnorr-fake}}$.

Honest-verifier zero-knowledge

A subtlety:

- Sigma protocols are “honest-verifier” ZK
- Assumes verifier follows the protocol
- Chooses challenges randomly
- Doesn't try to be clever

What about cheating verifiers?

- Might choose challenges adaptively
- Could try to extract information
- Harder to protect against!

Good news

For most applications, honest-verifier ZK is sufficient

Later: Fiat-Shamir

We'll see how to make protocols secure against any verifier by removing interaction entirely!

Putting it all together

A Sigma protocol is useful when it has all three properties:

Completeness

- Honest users succeed
- No false rejections
- Protocol is usable

Special Soundness

- Cheaters get caught
- Can extract witness
- Protocol is secure

Zero-Knowledge

- No information leaks
- Privacy preserved
- Protocol is private

Together, these properties create the “magic” of zero-knowledge proofs!

Section 3

Proving AND, OR

Combining proofs: AND and OR

What if we need to prove more complex statements?

OR statements:

- "I know the private key for A_1 OR A_2 "
- "This vote encrypts YES or NO"
- Prove one of several conditions
- Without revealing which one!

AND statements:

- "I know a AND b such that..."
- "These discrete logs are equal"
- Prove multiple conditions hold
- All must be satisfied

The challenge

How to combine Sigma protocols while preserving zero-knowledge?

Section 3

Proving AND, OR

Subsection 3.1

OR Proofs

Example use cases for OR proofs

1. Anonymous authentication:

- Public keys: A_1, A_2 (authorized users)
- Alice proves she knows one private key
- Doesn't reveal which user she is!

2. Electronic voting:

- Encrypted vote: $C = \text{Enc}(\text{vote})$
- Prove: $\text{Dec}(SK, C) \in \{\text{yes}, \text{no}\}$
- Ensures valid vote without revealing it

Key property

OR proofs hide *which* condition is satisfied

Privacy guarantee

Verifier learns that at least one condition holds, but not which one

The Robin Hood analogy

- *Robin Hood has two bows:*
 - One bow shoots accurately
 - One bow is erratic
 - King wants proof Robin can shoot well
 - But Robin doesn't want to reveal which bow works!
- *The challenge:*
 - King draws one target
 - Robin must fire both bows
 - Arrows must be equidistant from target
- *Robin's strategy:*
 1. Fire the erratic bow first (lands randomly)
 2. Calculate where to place second arrow
 3. Use accurate bow to hit that exact spot
 4. Target appears centered between arrows!
- **Control over one arrow is enough to satisfy the constraint**

OR proofs: the construction

Given: Sigma protocols Σ_0 for condition P_0 and Σ_1 for condition P_1

Goal: Prove “I know W such that $P_0(X, W)$ OR $P_1(X, W)$ ”

The protocol

1. **Commit:** Prover sends (Y_0, Y_1) - commitments for both protocols
2. **Challenge:** Verifier sends single challenge C
3. **Constraint:** Prover must provide (C_0, R_0) and (C_1, R_1) such that:
 - $C_0 + C_1 = C$ (challenges sum to verifier's challenge)
 - Both transcripts (Y_0, C_0, R_0) and (Y_1, C_1, R_1) are valid

OR proofs: how it works

If prover knows witness for P_0 :

1. Generate real Y_0 using witness
2. Simulate fake transcript for Σ_1 :
 - Choose random C_1, R_1
 - Compute Y_1 backwards
3. Receive challenge C from verifier
4. Set $C_0 = C - C_1$
5. Use witness to compute real R_0

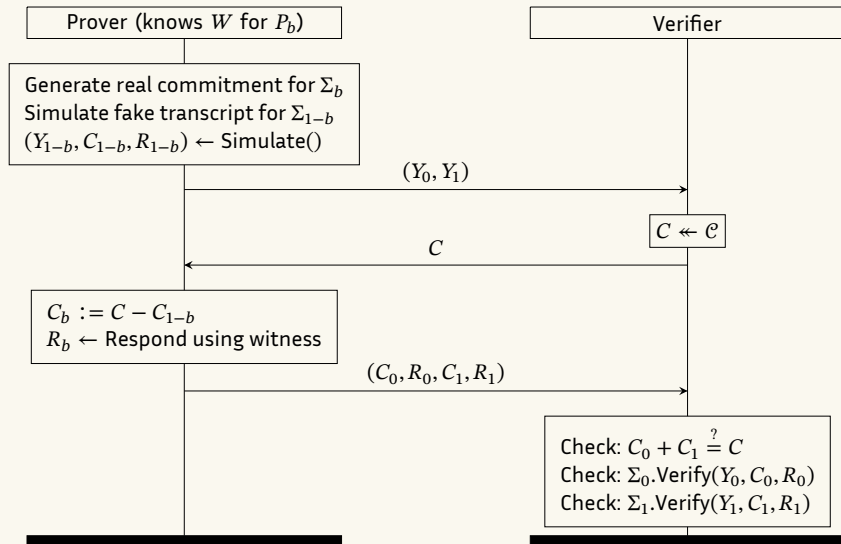
Why it's zero-knowledge:

- One transcript is real
- One transcript is simulated
- Verifier can't tell which is which!
- Distribution looks identical regardless of which witness we have

The trick

Prover controls one challenge completely, must adapt to the other

OR proof protocol diagram



Example: proving knowledge of one private key

Setup: Public keys $A_0 = g^{a_0}$ and $A_1 = g^{a_1}$

Alice knows a_0 (but not a_1) and wants to prove she knows one of them:

1. Commit phase:

- Real: Pick $y_0 \leftarrow \mathbb{Z}_n$, compute $Y_0 = g^{y_0}$
- Fake: Pick $C_1, R_1 \leftarrow \mathbb{Z}_n$, compute $Y_1 = g^{R_1} \cdot A_1^{-C_1}$
- Send (Y_0, Y_1)

2. Challenge: Receive C from verifier

3. Response phase:

- Compute $C_0 = C - C_1$
- Compute $R_0 = y_0 + C_0 \cdot a_0$
- Send (C_0, R_0, C_1, R_1)

Verification: Check $C_0 + C_1 = C$ and both Schnorr transcripts valid

Section 3

Proving AND, OR

Subsection 3.2

AND Proofs

AND proofs: proving multiple conditions

The problem:

- Prove multiple conditions simultaneously
- All must be true
- Example: "I know a such that $g_1^a = A_1$ AND $g_2^a = A_2$ AND $g_3^a = A_3$ "

Naive approach:

- Run protocols separately?
- Problem: different challenges
- Loses zero-knowledge!

The solution:

- Use the *same* challenge for all
- Commit to all statements
- Receive one challenge
- Respond to all with same challenge

How it works

Reusing the challenge maintains both soundness and zero-knowledge

AND proof construction

Given: Sigma protocols $\Sigma_1, \Sigma_2, \dots, \Sigma_k$ for conditions $P_1, P_2, \dots, P_k$

Goal: Prove "I know W such that $P_1(X, W)$ AND $P_2(X, W)$ AND ...AND $P_k(X, W)$ "

The protocol

1. **Commit:** Run commit phase for all protocols
 - $(Y_i, \sigma_i) \leftarrow \Sigma_i.\text{Commit}(X_i, W_i)$ for all i
 - Send $(Y_1, Y_2, \dots, Y_k)$
2. **Challenge:** Receive single C from verifier
3. **Respond:** Use same C for all protocols
 - $R_i \leftarrow \Sigma_i.\text{Respond}(\sigma_i, C)$ for all i
 - Send $(R_1, R_2, \dots, R_k)$
4. **Verify:** Check all transcripts (Y_i, C, R_i) are valid

Example: proving equality of discrete logs

Setup: Given $(g_1, g_2, g_3, A_1, A_2, A_3)$ where $A_i = g_i^a$ for same a

Protocol to prove knowledge of a :

1. Commit:

- Pick $y \leftarrow \mathbb{Z}_n$
- Compute $Y_1 = g_1^y$
- Compute $Y_2 = g_2^y$
- Compute $Y_3 = g_3^y$
- Send (Y_1, Y_2, Y_3)

2. Challenge: Receive C

3. Respond:

- Compute $R = y + C \cdot a$
- Send R

4. Verify:

- Check $g_1^R = Y_1 \cdot A_1^C$
- Check $g_2^R = Y_2 \cdot A_2^C$
- Check $g_3^R = Y_3 \cdot A_3^C$

Single response proves all!

One value R satisfies all three equations if and only if all discrete logs are equal

Combining AND and OR

We can build complex logical statements!

Example: “I know the private key for $(A_1 \text{ OR } A_2) \text{ AND } (B_1 \text{ OR } B_2)$ ”

- Meaning: I’m one of users $\{A_1, A_2\}$ AND one of users $\{B_1, B_2\}$
- Four possible combinations, but verifier learns nothing about which!

Construction approach:

1. Build OR protocols for each clause
2. Combine them using AND construction
3. Result: Single protocol proving the entire statement

Section 4

Non-Interactive Zero-Knowledge Proofs

Section 4

Non-Interactive Zero-Knowledge Proofs

Subsection 4.1

Fiat-Shamir

Non-interactive proofs: removing the verifier

The limitation of interactive proofs

Interactive Sigma protocols:

- Require back-and-forth communication
- Prover and verifier must be online together
- Can't post a proof for later verification
- Can't broadcast a proof to many verifiers

What we want:

- Prover generates proof alone
- Anyone can verify later
- No interaction needed
- But still zero-knowledge!

The challenge

How to ensure C comes after Y without a verifier?

The Fiat-Shamir transformation

Replace the verifier with a hash function!

- **Requirements:**

- Challenge must be unpredictable
- Must depend on commitment Y
- Solution: $C := \text{HASH}(X, Y)$
- Hash function acts as “virtual verifier”

- **Properties needed:**

- H modeled as random oracle
- Output unpredictable before input
- Uniformly distributed output

Random oracle

- A theoretical hash function that:
 - Returns random output for each new input
 - Always returns same output for same input
 - Output is uniformly distributed
- In practice, we use SHA-256 or similar, which approximates these properties

Fiat-Shamir: the construction

Given a Sigma protocol Σ , create a non-interactive proof system:

Prove(X, W):

1. $(Y, \sigma) := \Sigma.\text{Commit}(X, W)$
2. $C := \text{HASH}(X, Y)$
3. $R := \Sigma.\text{Respond}(\sigma, C)$
4. Return (Y, R)

Verify(X, Y, R):

1. $C := \text{HASH}(X, Y)$
2. Return $\Sigma.\text{Check}(X, Y, C, R)$

The proof is just (Y, R) - no interaction needed!

Example: Non-interactive Schnorr

Interactive Schnorr:

1. Prover sends $Y = g^y$
2. Verifier sends random c
3. Prover sends $r = y + ca$
4. Verifier checks $g^r = Y \cdot A^c$

Fiat-Shamir Schnorr:

1. Prover computes $Y = g^y$
2. Prover computes $c = \text{HASH}(A, Y)$
3. Prover computes $r = y + ca$
4. Proof is (Y, r)
5. Verifier recomputes $c = \text{HASH}(A, Y)$
6. Verifier checks $g^r = Y \cdot A^c$

This is a digital signature!

If we include a message m in the hash: $c = \text{HASH}(A, Y, m)$, we get the Schnorr signature scheme!

Why Fiat-Shamir is sound

The order of operations is crucial

- **With random oracle:**
 - Must choose Y first
 - Then compute $C = \text{HASH}(X, Y)$
 - Can't predict C before choosing Y
 - Can't "work backwards" like in simulation
- **This enforces honest behavior:**
 - Prover commits before seeing challenge
 - Just like in interactive protocol
 - Hash function enforces ordering

Key difference from interactive

- In interactive Sigma protocol:
 - Can simulate by choosing C, R first
 - Then compute Y backwards
 - This creates deniability
- In Fiat-Shamir:
 - Can't choose C arbitrarily
 - Must be $C = \text{HASH}(X, Y)$
 - Can't work backwards!
 - **Loses deniability property**

The Forking Lemma: extracting witnesses

Lemma (Forking Lemma)

If an algorithm A can produce valid Fiat-Shamir proofs with non-negligible probability, then we can extract a valid witness with non-negligible probability.

The extraction strategy:

1. Run A until it outputs valid proof (Y, R)
2. "Rewind" A to when it queried $\text{HASH}(X, Y)$
3. Give a different hash output C' this time
4. If A outputs another valid proof with same Y :
 - We have two transcripts: (Y, C, R) and (Y, C', R')
 - Extract witness using special soundness!

Wait, how can the hash give different outputs?

If $\text{HASH}(X, Y)$ is deterministic, how do we get $C' \neq C$?

- We're in the **random oracle model**
- The extractor *controls* the oracle
- We can “reprogram” hash outputs!
- Same input, different output (in simulation)

How rewinding works:

1. Run algorithm A with oracle H
2. Record query $\text{HASH}(X, Y) = C$
3. Rewind to that point
4. Answer with $C' \neq C$ instead

This is a proof technique

- Not how real hashes work!
- Extractor is a *thought experiment*
- Shows: if you can forge proofs, you must know the witness
- Real hash functions are deterministic

If A succeeds for *any* hash output, it must be able to handle multiple challenges $\Rightarrow$ extraction works

Deniability lost!

Fiat-Shamir destroys the deniability property

Interactive Sigma protocol:

- Transcripts are deniable
- Anyone can generate fake transcripts
- Choose C, R first, compute Y
- Indistinguishable from real
- **Can't prove to third parties**

Fiat-Shamir proof:

- Have constraint $C = \text{HASH}(X, Y)$
- Everyone can verify this constraint
- Can't generate fake proofs
- Only witness holder can create
- **Transferable evidence!**

Trade-off

We gain non-interactivity but lose deniability. The proof becomes a permanent, transferable piece of evidence.

Applications of Fiat-Shamir

Fiat-Shamir is widely used in practice

Digital signatures:

- Schnorr signatures
- EdDSA (used in many systems)
- ECDSA (related derivation from ElGamal)

Blockchain applications:

- Proof of reserves
- Ring signatures for privacy
- Bulletproofs (range proofs)

Advanced protocols:

- SNARKs (succinct proofs)
- STARKs (transparent proofs)
- Proof of knowledge of signature

Key advantage

Convert any interactive ZK proof into a non-interactive one that can be verified by anyone, anytime

Section 4

Non-Interactive Zero-Knowledge Proofs

Subsection 4.2

Fiat-Shamir Circuits

From statements to circuits: a paradigm shift

Wait, why are we suddenly talking about circuits?

So far, we proved statements like:

- "I know a such that $g^a = A$ "
- "I know a 3-coloring of this graph"
- Specific mathematical relations

But what if we want to prove:

- "I computed this SHA-256 correctly"
- "This transaction is valid"
- "I executed this program correctly"

The idea

Any computation can be represented as a circuit!

Think of it like:

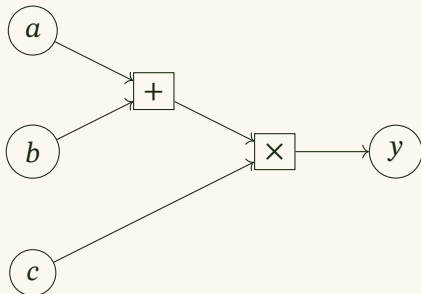
Instead of proving "I know x ", we prove "I know x such that circuit $C(x) = y$ "

What is a circuit?

- **A circuit is just a computation graph:**

- Gates: AND, OR, NOT, ADD, MUL, etc.
- Wires: Connect gates together
- Inputs: Public (x) and private (w)
- Output: Result of computation

- **Example:** $y = (a + b) \cdot c$



- **Why circuits?**

- Universal representation
- Any algorithm $\rightarrow$ circuit
- Fixed structure (no loops!)
- Easy to verify gate-by-gate

The power

Once we can prove statements about circuits, we can prove statements about *any computation!*

The zero-knowledge circuit paradigm

The fundamental shift in thinking

Old way (Sigma protocols):

- Design specific protocol for each statement
- Schnorr for discrete log
- Different protocol for factoring
- Custom protocol for each problem

New way (Circuit ZK):

- Convert problem to circuit
- Use general ZK protocol for circuits
- Same protocol works for everything!
- Just change the circuit

This changes everything!

Instead of crafting clever protocols, we write programs and prove we executed them correctly

Example: proving knowledge of a hash preimage

Claim: “I know x such that $\text{SHA256}(x) = y$ ”

As a Sigma protocol:

- Would be incredibly complex
- SHA-256 has 64 rounds
- Each round has bitwise operations
- No elegant mathematical structure
- Basically impossible in practice!

As a circuit:

- Write SHA-256 as circuit
- Input: private x
- Output: public y
- Apply general circuit ZK protocol
- Done! Works automatically

From theory to practice: ZK programming

Modern ZK development:

- Write programs in high-level languages
- Compiler converts to circuits
- ZK protocol proves circuit execution
- Developer doesn't see the circuit!

Example (Circom syntax):

```
template IsZero() {  
  signal input in;  
  signal output out;  
  // Proves: out = (in == 0)  
}
```

The compilation pipeline:

1. High-level code
2. Intermediate representation
3. Arithmetic circuit
4. Constraint system
5. ZK proof

We've come full circle

Started with "I know a secret"
Now: "I ran this program correctly"
Both use the same ZK machinery!

The cost of generality

Circuit ZK is powerful but comes with trade-offs

Advantages:

- Universal - works for any computation
- Compositional - combine circuits
- Tool support - compilers, libraries
- No need for crypto expertise

Disadvantages:

- Less efficient than custom protocols
- Circuit can be huge (millions of gates)
- Proving time can be slow
- More complex security analysis

Rule of thumb

Use custom Sigma protocols for simple statements (like Schnorr)

Use circuit ZK for complex computations (like program execution)

Section 4

Non-Interactive Zero-Knowledge Proofs

Subsection 4.3

Attacks on Fiat-Shamir

The problem with Fiat-Shamir

Can we actually trust Fiat-Shamir in practice?

The promise:

- FS is secure in random oracle model
- Replace oracle with concrete hash
- Should be secure in practice

Known theoretical attacks:

- Canetti-Goldreich-Halevi (2004)
- Barak (2001), Goldwasser-Kalai (2003)
- But all use contrived protocols!

New result (2025)

First practical attack on a standard protocol!^a

- Targets GKR-based proof systems
- Used in real deployed systems
- Attack works for *any* hash function

^a<https://appliedcryptography.page/paper/#prove-false>

The GKR protocol under attack

What is GKR?

- Protocol by Goldwasser-Kalai-Rothblum
- Proves correctness of computations
- Works for bounded-depth circuits
- Combined with polynomial commitments

The claim being proved:

- Circuit C , input x , witness w
- Prove: $\exists w$ such that $C(x, w) = y$

Why this matters:

- Widely studied in literature
- Used in practice (e.g., Expander)
- Considered secure with Fiat-Shamir
- Foundation of many SNARKs

The protocol flow

1. Commit to witness w
2. Verifier picks random r
3. Run GKR reduction
4. Verify with polynomial commitment

The attack: exploiting self-reference

The circular dependency problem:

- Circuit C^* determines challenge r
- Changing C^* changes r
- How to break this cycle?

The solution: self-describing circuits

- Give the circuit its own description!
- Witness $w = \langle C^* \rangle$ (circuit digest)
- Now circuit can compute r internally
- No circular dependency

The trick

Circuit outputs $(\gamma, \gamma - 1)$ where $\gamma = \text{HASH}(\langle C^* \rangle, y^*, \text{comm}(w))$. Claim output is $(0, 0)$ — always false!

What is a circuit digest?

From circuit to digest:

- A circuit C is a computation graph
- Gates, wires, input/output structure
- Can be serialized to bytes
- **Circuit digest:** $\psi = \langle C \rangle$

The encoding process:

1. Enumerate all gates in order
2. Record gate types and connections
3. Include input/output specifications
4. Hash or encode to fixed-size digest

Why digests matter for the attack:

- Circuit can “know itself”
- Pass $\psi = \langle C^* \rangle$ as witness
- Circuit computes hash of own description
- Creates self-reference needed for attack

If a circuit is deep enough, it can:

- Take its own digest as input
- Compute $\text{HASH}(\psi, \dots)$
- Predict its own challenge!

How the attack works step by step

The attacking circuit C^* :

1. Input: witness w
2. Interpret w as circuit digest ψ
3. Compute $\alpha = \text{comm}(w)$
4. Compute $\gamma = \text{HASH}(\psi, y^*, \alpha)$
5. Output: $(\gamma, \gamma - 1)$

The false claim:

- Prover claims: $\exists w : C^*(w) = (0, 0)$
- Reality: Output is always $(\gamma, \gamma - 1) \neq (0, 0)$

Why verification passes:

- Set $w = \langle C^* \rangle$
- Verifier computes:
 $r = \text{HASH}(\langle C^* \rangle, y^*, \alpha)$
- But this equals γ by construction!
- MLE of real output at r :
 $\hat{y}(r) = \gamma - r = 0$
- MLE of claimed $(0, 0)$ at r : also 0
- They match! GKR proceeds honestly from here

At exactly the challenge point r , real and fake outputs have the same MLE value

Understanding the attack: a real-world analogy

Imagine a magic lock:

- The lock asks you a random question
- You must answer correctly to open it
- The question depends on what you show it
- Normally: you can't predict the question

The honest scenario:

1. Show the lock your credentials
2. Lock generates random challenge
3. You answer based on your secret
4. Lock verifies and opens

The attack:

- You build a special mirror
- Mirror shows the lock... itself!
- Lock's question now depends on the mirror
- But mirror was built knowing the lock
- You pre-computed the "random" question!

The attack analogy continued: exam cheating

Think of it like an exam:

- Professor creates exam based on submitted essay
- Questions are “random” but derived from your essay
- Write essay first, then face unknown questions

The cheater's trick:

1. Submit an essay on “how this exam will be graded”
2. Essay contains the grading algorithm itself
3. Essay predicts exactly which questions will be asked
4. Cheater embeds the answers in the essay!

Why it works:

- “Random” challenge comes from hashing your input
- But your input describes the hash process
- You compute the hash inside your submission
- Your answer is tailored to that exact challenge

Why this attack is devastating

What makes this different:

- GKR is widely used
- Works for *any* hash
- Affects real deployed systems

The depth requirement:

- Attack needs: $d \geq d_{\text{comm}} + d_h + O(1)$
- Circuit must be deep enough to compute hash + commitment
- Many practical circuits satisfy this!

Key properties exploited:

- GKR doesn't commit to computation trace
- Challenge point determined by hash of transcript
- Circuit can embed cryptographic operations
- Self-reference through witness

Extended attack: backdooring any circuit

We can insert a backdoor into ANY circuit!

Given:

- Any circuit $C(x, w) \rightarrow y$
- Desired input x^* and output y^*

We construct:

- Circuit C^* functionally equivalent to C
- But can prove $C^*(x^*, w) = y^*$
- Even if this is FALSE!

The backdoored circuit:

1. Compute $y = C(x, w)$ normally
2. Check if conditions match attack
3. If yes, manipulate output
4. Otherwise, output y

Critical insight

Security depends on circuit
implementation, not just functionality!

Implications for practice

What does this mean for real systems?

Immediate impact:

- GKR+FS is not secure in general
- Affects deployed systems
- No “safe” hash function choice
- Circuit implementation matters!

Particularly concerning:

- Recursive proof composition
- Circuits that use crypto primitives
- Hard to audit implementations

Mitigations:

- Restrict circuit depth
- Avoid circuits with crypto ops
- Use canonical representations
- Domain separation techniques

Silver lining

Simple circuits (e.g., for lookups) seem safe - they have canonical forms

Section 5

zkVMs

What is a zkVM?

- **The concept:**

- A black-box virtual machine that executes programs
- Generates ZK proofs of correct execution
- Proves: "I ran program P on input X and got output Y"
- *Without revealing intermediate states!*

- **Think of it as:**

- Computer + ZK proof system
- Run any program privately
- Prove the result is correct

- **Why is this revolutionary?**

- No need to design custom circuits
- Write normal code, get ZK proofs
- Developers don't need crypto expertise
- One system for all computations

zkVM architecture and components

Core components:

- **Instruction Set:**
 - RISC-V, MIPS, or custom
 - Simple, well-defined operations
 - Each instruction becomes constraints
- **Memory model:**
 - RAM with read/write operations
 - Stack and heap management
 - Merkle trees for efficiency
- **Execution trace:**
 - Record of all state transitions
 - Forms the witness for the proof

The proving process:

1. Load program into VM
2. Execute with private inputs
3. Record execution trace
4. Generate constraints from trace
5. Create ZK proof of constraints
6. Output: proof + public result

Examples in practice

RISC Zero, SP1, Jolt, Cairo VM - each with different trade-offs

zkVM applications and impact

- **Blockchain scaling:**

- Rollups: execute off-chain, prove on-chain
- Compress thousands of transactions
- Ethereum's Layer 2 revolution
- Orders of magnitude cheaper

- **Verifiable computation:**

- Cloud computing with proof
- Outsource work, verify results
- No trust in compute provider
- Perfect for untrusted environments

- **Private applications:**

- Machine learning inference
- Private smart contracts
- Confidential databases
- Regulatory compliance proofs

The trade-off

Proving is 1000-1,000,000× slower than native execution

Section 5

zkVMs

Subsection 5.1

RISC Zero

Remember this?

- **Zero-knowledge proofs** allow you to prove that you know a secret without revealing any information about it.
- They built “zero-knowledge virtual machines” where you can execute an entire program that runs as a zero-knowledge proof.
- ZKP battleship game: server proves to the players that its output to their battleship guesses is correct, without revealing any additional information (e.g. ship location).



Battleship board game. Source: Hasbro

RISC Zero: making ZK development accessible

- **What it offers:**

- Write ZK apps in Rust/C++
- No circuit design needed
- No custom languages
- Leverage existing ecosystem
- 70% of top Rust crates work!

- **Key use cases:**^a

- ZK Coprocessors for blockchains
- Optimistic rollups with ZK fraud proofs
- Verifiable off-chain computation

- **Example applications built:**

- **Zeth:** Prove correct Ethereum blocks
- **Bonsai Pay:** Send ETH to email addresses
- **JSON:** Selective disclosure of data
- **Where's Waldo:** Prove without revealing
- **ZK Checkmate:** Prove mate-in-one without revealing
- **Proof of Exploit:** Responsible disclosure

^a<https://dev.risczero.com/api/use-cases>

Getting started with RISC Zero

- **Building a zkVM application:^a**

1. Write guest program in Rust
2. Compile to ELF binary
3. Execute and record session
4. Generate proof (receipt)
5. Anyone can verify with ImageID

- **The guest program:**

- Core logic that gets proven
- Standard Rust code
- Access to most crates
- Defines public/private inputs

- **Deployment options:**

- **Local proving:**

- Need 16GB+ RAM
- GPU acceleration available
- Full control over process

- **Bonsai (remote):**

- Cloud-based proving
- No host code needed
- Pay per proof
- Easier to get started

^a<https://dev.risczero.com/api/zkvm/quickstart>

ZK Battleship: proving without revealing

The classic problem:

- Player 1 (server) has a secret board
- Player 2 makes guesses (shots)
- Server responds: hit, miss, sunk
- How can Player 2 trust the responses?

Traditional solutions:

- Trusted third party
- Reveal board at end

The zkVM solution:

- Server proves each response is correct
- Without revealing ship positions!
- Player 2 verifies proof cryptographically
- Cheating is mathematically impossible

The witness

The secret GameState: 5 ships with positions, orientations, and a random “pepper” for hiding

ZK Battleship: the initialization proof

Step 1: Prove valid board setup

- Server commits to initial board state
- Proves all ships are placed correctly:
 - All 5 ships present (correct types)
 - Ships within 10×10 bounds
 - No ships overlap
- Outputs only a SHA-256 commitment hash

What verifier learns

“Server has a valid board configuration”
Nothing about where ships are located!

The guest program (init.rs):

```
fn main() {  
    // Read secret state  
    let state: GameState =  
        env::read();  
    // Validate the board  
    if !state.check() {  
        panic!("Invalid!");  
    }  
    // Output commitment  
    env::commit(&state.commit());  
}
```


ZK Battleship: the round proof

Step 2: Prove each shot response

- For each shot, server proves:
 1. Started with committed state
 2. Applied the shot correctly
 3. Computed right result (hit/miss/sunk)
 4. Produced valid new state
- Outputs: old commit, new commit, shot, result

Chain of commitment: prevents tampering

Each round's `old_state` must match previous round's `new_state`

The guest program (round.rs):

```
fn main() {  
    let old = state.commit();  
    let hit =  
        state.apply_shot(shot);  
    let new = state.commit();  
    env::commit(&RoundCommit {  
        old_state: old,  
        new_state: new,  
        shot, hit,  
    });  
}
```

ZK Battleship: verification and security

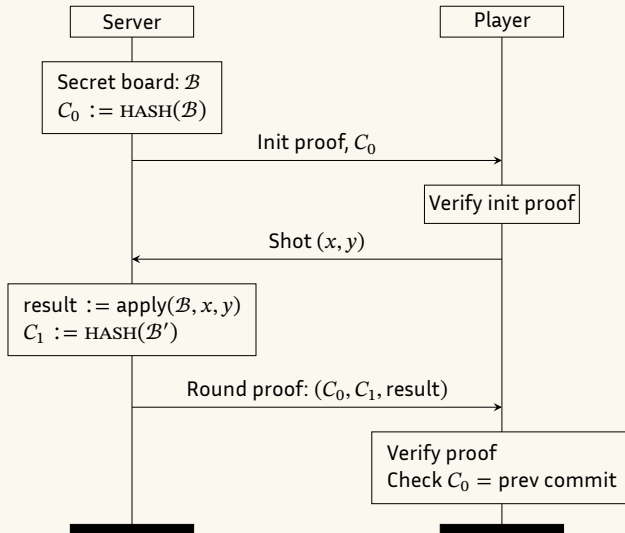
What the host verifies:

1. **Init proof valid:** Board follows game rules
2. **Round proof valid:** Shot computed correctly
3. **State consistency:** `old_state` matches previous `new_state`
4. **Shot consistency:** Proof uses the actual shot sent

Security guarantees:

- **Privacy:** Board never revealed to verifier
- **Soundness:** Cannot lie about hit/miss results
- **Integrity:** Cannot change board mid-game

ZK Battleship: verification and security



Section 5

zkVMs

Subsection 5.2

Making zkVMs Practical

The zkVM promise vs. reality

- **The promise:**

- “Democratize SNARKs”
- No specialized expertise needed
- Prove any program execution
- Amazing developer experience
- Ready for deployment today

- **The reality:**

- Riddled with bugs
- 100,000× slower than native
- Years from production-ready
- Security theater, not security

- **The hard truth:**

- Projects claiming to use zkVMs for security today are either:
 - Secured by permissioning (not SNARKs)
 - Vulnerable to attack
 - Paying enormous costs for false security

- **Time to maturity:**

- We are still **years away** from meeting even basic goals for secure and performant zkVMs

Security stages: the path to trusted zkVMs

- **Stage 1: Correct protocols**

- **1a:** Formally verify PIOP soundness
- **1b:** Verify PCS is binding
- **1c:** Verify Fiat-Shamir security
- **1d:** Verify constraint $\leftrightarrow$ VM semantics
- **1e:** Glue everything together

- **Timeline:** No zkVM likely to fully meet Stage 1 for at least **2 years**

- **Stage 2: Correct verifier**

- Formally verify implementation
- Match protocol from Stage 1
- Focus on verifier (simpler than prover)
- Ensures soundness in practice

- **Stage 3: Correct prover**

- Verify prover implementation
- Ensures completeness
- Zero-knowledge if claimed

- **Timeline:** Stage 3 in **4+ years**

Critical caveats and open problems

- **Fiat-Shamir vulnerability:**

- Open research questions remain
- Random oracle $\neq$ real hash functions
- Known attacks involve recursive circuits
- Stage 2 system could be found insecure!

- **The bytecode problem:**

- Proving correct execution of flawed code?
- Need formally verified bytecode
- Enormous challenge outside zkVM scope

- **Recursion complications:**

- Every PIOP must be verified
- Every commitment scheme too
- Every constraint system
- Multiplies verification burden

- Current deployments skimp on security for performance:

- Some use <100 bits security
- Should use 128+ bits minimum
- If performance were good, we wouldn't compromise

Performance reality check

- **Current performance:**

- 100,000× to 1,000,000× slower
- Simple operations take seconds
- Complex programs take hours
- Memory operations particularly costly

- **Why so slow?**

- Every instruction → constraints
- Memory access needs Merkle proofs
- Polynomial commitments are expensive
- Recursion adds more overhead

- **The performance stages ahead:**

- Hardware acceleration (GPUs, ASICs)
- Better constraint systems
- Optimized polynomial arithmetic
- Proof composition techniques

Reality check

Even with all optimizations, expect 1000× overhead for years to come

The road ahead: realistic expectations

- **What to focus on now:**

- Formal verification efforts
- Basic security properties
- Performance fundamentals
- Not post-quantum (yet)
 - Quantum computers: 5+ years away
 - Bugs: existential risk today
 - Use classical SNARKs for now

- **Realistic timeline:**

- 2 years: Stage 1 protocols
- 3 years: Stage 2 verifiers
- 4+ years: Stage 3 provers
- 5+ years: Production-ready

- **Key takeaways:**

- Don't believe the hype
- Current deployments are insecure
- Performance is prohibitive
- Years of work ahead

Remember

Anyone claiming zkVMs are ready for production deployment is either:

- Misinformed
- Misleading you
- Taking enormous risks

Section 6

Zero-Knowledge Proofs in the Real World

Section 6

Zero-Knowledge Proofs in the Real World

Subsection 6.1

Zcash: Privacy-Preserving Cryptocurrency

Zcash: bringing ZK to cryptocurrency

- **Bitcoin's transparency:**

- All transactions are public
- Addresses are pseudonymous
- But can be linked and traced
- Chain analysis reveals patterns
- Privacy through obscurity fails

- **The need for privacy:**

- Financial privacy is fundamental
- Fungibility requires privacy
- Businesses need confidentiality

- **Enter Zcash (2016):**

- First production use of zk-SNARKs
- Optional privacy for transactions
- Prove transaction validity without revealing:
 - Sender
 - Receiver
 - Amount
- Revolutionary achievement!

How Zcash works: shielded transactions

Two types of addresses:

- **Transparent (t-addresses):**
 - Work like Bitcoin
 - Fully visible on blockchain
 - Start with 't'
- **Shielded (z-addresses):**
 - Use zero-knowledge proofs
 - Hide transaction details
 - Start with 'z'

What the ZK proof proves:

- Input notes exist
- Sender owns the input notes
- No double spending

The magic

All this is proved without revealing which notes are spent or created!

Evolution and improvements

Performance journey:

- **Sprout (2016):**
 - Proof generation: 40 seconds
 - Memory: 3 GB
 - Mobile impossible
- **Sapling (2018):**
 - Proof generation: 2.5 seconds
 - Memory: 40 MB
 - Mobile feasible
- **Halo 2 (Today):**
 - No trusted setup!
 - Recursive proofs
 - Even better performance

Key innovations:

- Viewing keys for auditing
- Memo field for encrypted messages
- Unified addresses
- Hardware wallet support

The trade-off

Privacy comes at a cost: shielded transactions are more expensive and complex than transparent ones

Impact and lessons learned

Technical achievements:

- First production ZK system at scale
- Proved ZK can work in adversarial environment
- Inspired countless other projects
- Advanced the field significantly

Challenges faced:

- Regulatory concerns
- Exchange delistings
- Slow progress and growth

Lessons for ZK adoption:

- Privacy must be easy to use
- Optional privacy → limited adoption
- Performance matters enormously
- Trust assumptions are critical
- Regulatory environment is key

Legacy

Zcash showed the world that zero-knowledge proofs aren't just theory - they can secure billions in value

Section 6

Zero-Knowledge Proofs in the Real World

Subsection 6.2

ZK Goes Mainstream: Google's Open Source Initiative

From academic curiosity to industry adoption

What just happened?

- Google open-sourced ZKP libraries
- Partnership with Sparkasse (German bank)
- Focus on EU age assurance
- Code available on GitHub^a

Why this matters:

- Tech giant embracing privacy tech
- Real-world deployment at scale
- Not just research anymore!

The promise fulfilled

“In layperson’s terms, ZKP makes it possible for people to prove that something about them is true without exchanging any other data.”

Think about it

We’ve spent this lecture learning the theory. Now it’s becoming everyday technology!

^a<https://github.com/google/longfellow-zk>

The age verification problem

Current approaches:

- Upload government ID
- Credit card verification
- Face recognition + AI
- Third-party identity services

Problems:

- Privacy nightmare
- Data breaches
- Identity theft risk
- Tracking across sites

The ZK solution:

- Prove: "I am over 18"
- Reveal: Nothing else!
- No name, no birthdate
- No tracking possible
- Cryptographically secure

Question for you

What other attributes could we prove this way? Think beyond age...

The European regulatory landscape

eIDAS Regulation (2026):

- European Digital Identity Wallet
- Every EU citizen gets one
- Must work across all member states
- Privacy-enhancing tech encouraged

The tension:

- Governments want: Control, compliance
- Citizens want: Privacy, freedom
- ZK offers: Both! Or does it?

Critical thinking

- Is this true privacy protection?
- Who controls the infrastructure?
- Can governments revoke proofs?
- What about metadata leakage?

Looking for extra credit?

Research the EUDI Wallet proposal. Does it truly protect privacy or just shift trust?

The open source angle: democratizing privacy

Benefits of open sourcing:

- Transparency - no backdoors
- Community review
- Faster innovation
- No vendor lock-in
- Educational value

The ecosystem effect:

- Developers can build on it
- Researchers can improve it
- Standards can emerge

But consider:

- Google's motivations?
- Control through contribution?
- Patent considerations?
- Implementation vs. specification

Section 6

Zero-Knowledge Proofs in the Real World

Subsection 6.3

The Future of Real-World Zero-Knowledge Systems

Beyond age verification: the possibilities

Where else could ZK proofs transform our digital lives?

Financial services:

- Prove income without revealing amount
- Creditworthiness without credit score
- KYC without identity exposure

Healthcare:

- Vaccination status
- Insurance eligibility
- Clinical trial participation

Social platforms:

- Prove you're human (not bot)
- Community membership
- Reputation without identity

Government services:

- Voting eligibility
- Benefit qualification
- License verification

Technical challenges ahead

Current limitations:

- Proof generation time
- Verification complexity
- Key management
- Trusted setup ceremonies
- Quantum resistance

Research frontiers:

- Recursive proof composition
- Hardware acceleration
- Post-quantum ZK
- Universal circuits

User experience challenges:

- How to explain ZK to users?
- Wallet backup and recovery
- Cross-device synchronization
- Fallback mechanisms

The future of privacy online

Three possible futures - which will we choose?

Surveillance capitalism

- Status quo continues
- Data = currency
- Privacy theater
- ZK never adopted

Privacy renaissance

- ZK everywhere
- Self-sovereign identity
- Minimal disclosure
- User empowerment

Hybrid reality

- Some privacy gains
- Corporate control
- Government oversight
- Compromises

Questions to ponder

1. **Trust models:** If Google controls the ZK infrastructure, have we really gained privacy or just shifted trust?
2. **Adoption barriers:** What will it take for average users to understand and demand ZK proofs?
3. **Economic incentives:** Who profits from privacy? Who loses? How does this affect adoption?
4. **Global implications:** How will authoritarian regimes respond to unbreakable privacy tech?
5. **Your role:** Will you be a builder, a critic, a user, or an advocate of this technology?



Applied Cryptography

Summer 2026

Part 2: Real-World Cryptography

2.8: Zero-Knowledge Proofs

Nadim Kobeissi

<https://appliedcryptography.page>